

МОСКОВСКИЙ АВИАЦИОННЫЙ ИНСТИТУТ
(НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ)

Институт №8 «Компьютерные науки и прикладная математика»
Кафедра 806 «Вычислительная математика и программирование»

**Лабораторная работа №1
по курсу “Компьютерная графика”**

Основы 3D графики

Выполнил: Д. А. Алгиничев
Группа: М8О-308БВ-24
Преподаватель: В. Д. Бахарев

Москва, 2026

Условие

Построение простых 3D-объектов, проекцией на 2D-плоскость. Научиться работать с матрицами перспективы, ортогографической проекции и аффинными преобразованиями.

Вариант 1. Куб.

Метод решения

1. Инициализация ресурсов Vulkan.

Через VMA создаются `VkBuffer` для вершин и индексов куба, а также три `uniformBuffers` под UBO. Для них создаётся `VkDescriptorSetLayout`, `VkDescriptorPool` и три `VkDescriptorSet`. Затем собирается `VkPipelineLayout` и `VkPipeline` с вершинным и фрагментным шейдерами, `depth`-тестом и динамическими `viewport/scissor`.

2. Математика сцены.

Для каждого куба модельная матрица считается как

$$M = T(\text{pos}) * R_x * R_y * R_z * S(\text{scale})$$

Через `glm::translate`, `glm::rotate`, `glm::scale`.

Видовая матрица задается `glm::lookAt(eye, center, up)`.

Проекция выбирается переключателем:

`glm::perspective(pi/4, aspect, 0.1, 100)` или `glm::ortho(...)`.

Для Vulkan выполняется `proj[1][1] *= -1.0f` (из-за особенности направления осей `x` и `y` в Vulkan).

Анимация кубов - движение по окружности:

$$\text{pos.x} = \cos(t) * R, \text{pos.z} = \sin(t) * R, \text{плюс вращение по X и Y.}$$

3. Обновление сцены.

В `update()` через `ImGui` меняются позиции, поворот, масштаб, цвет и параметры анимации. Затем для трёх объектов (два куба и пол) заполняются UBO: `model`, `view`, `proj`, `baseColor`.

Данные копируются в отображённую память `uniformBuffersMapped[i]`.

4. Рендер.

В `render()` записывается `command buffer`:

начинается `render pass` с очисткой цвета и глубины, биндится `graphicsPipeline`, задаются `viewport` и `scissor`, биндятся общие `vertexBuffer` и `indexBuffer`. Для каждого из трёх объектов биндится

свой descriptorSet и вызывается vkCmdDrawIndexed. В конце render pass и command buffer завершаются.

5. Завершение.

В shutdown() уничтожаются pipeline, layout, descriptor pool/layout и все буферы через vmaDestroyBuffer.

Результаты

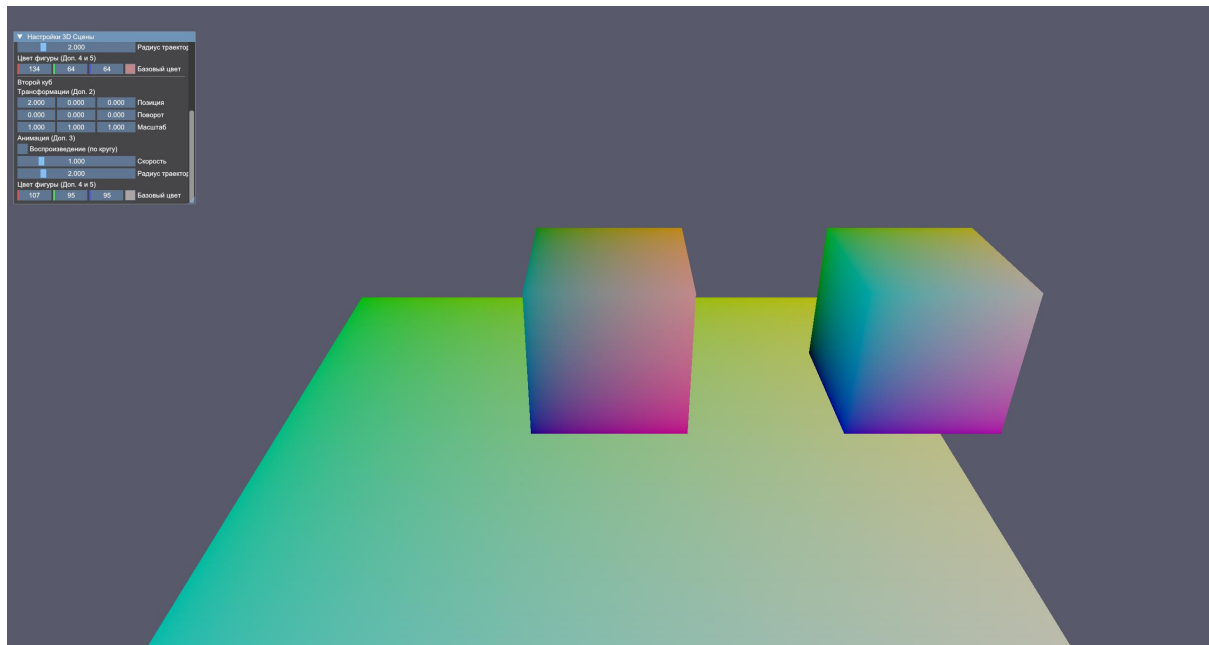


Рис 1. Вид на фигуры в перспективной проекции

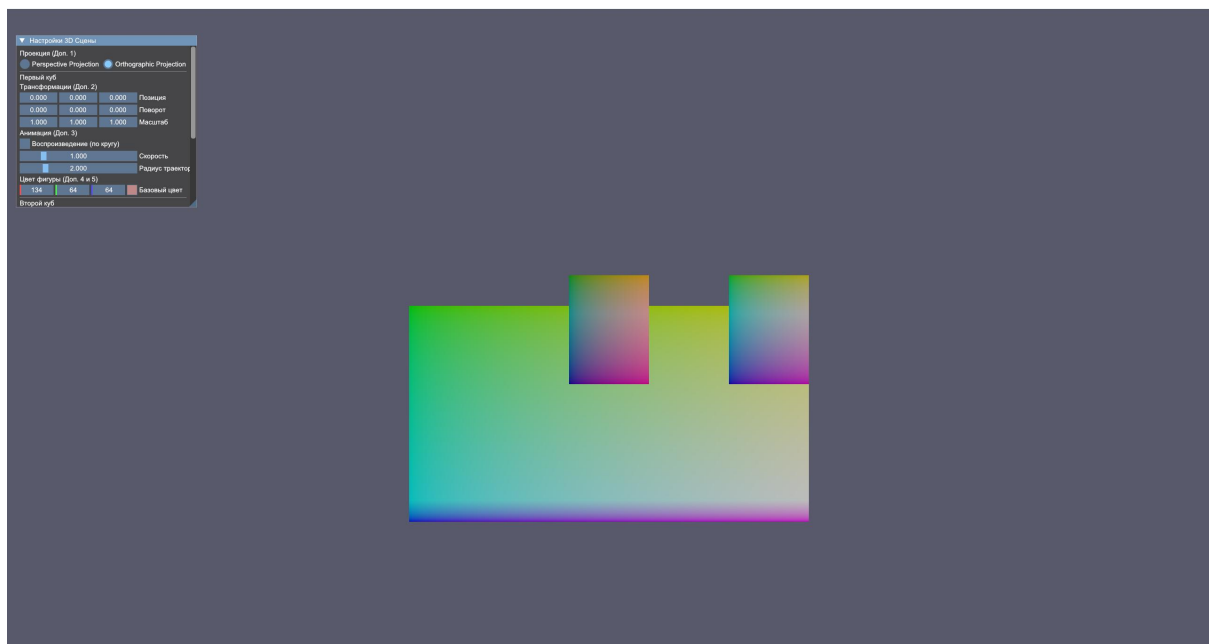


Рис 2. Вид фигур в ортогографической проекции

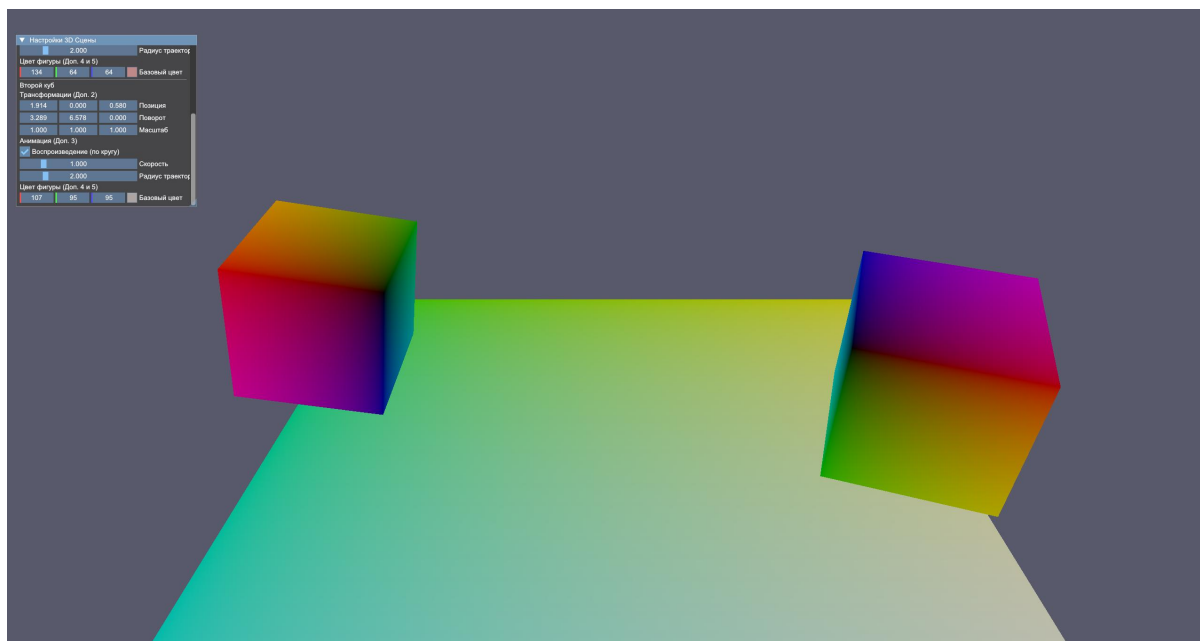


Рис 3. Вид в фигур во время анимации

Выводы

В ходе выполнения лабораторной работы, я научился строить простые 3D-объекты проекцией на 2D-плоскость. Научился работать с матрицами перспективы, ортогографической проекции и аффинными преобразованиями.